

Laboratorio de Computación

Salas A y B

Profesor(a): Carlos Chaves Mercado

Asignatura: Fundamentos de programación

Grupo: 8

No de Práctica(s): Práctica de estudio 10: Arreglos multidimensionales

Integrante(s): Nolasco Ramírez Sofia

Macay Martínez David

No. de lista o brigada: Mesa 6

Semestre: 2026-2

Fecha de entrega: 1 de mayo de 2026

Observaciones:

CALIFICACIÓN: _____

Objetivo:

El alumno utilizará arreglos de dos dimensiones en la elaboración de programas que resuelvan problemas que requieran agrupar datos del mismo tipo, en estructuras que utilizan dos índices.

Introducción:

En la resolución de problemas mediante el uso de programación estructurada, existen ocasiones en las que una organización lineal de datos resulta ser insuficiente para representar lo necesario. Por un lado, existen los vectores que permiten manejar listas de elementos; los arreglos bidimensionales o matrices introducen otra dimensión lógica que permite organizar información en esquemas de filas y columnas.

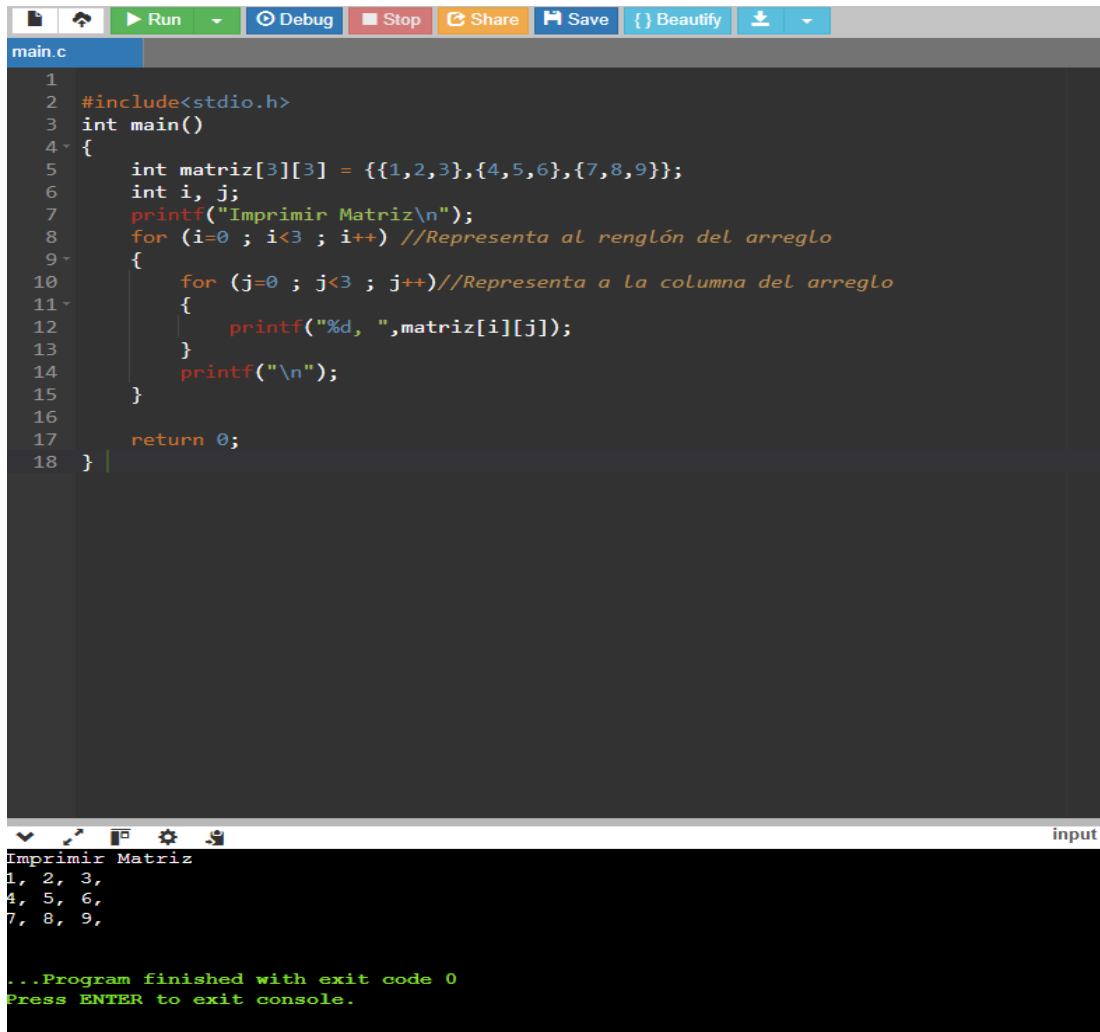
Una matriz es una colección finita y homogénea de datos donde cada elemento se identifica mediante 2 índices: primero la fila y segundo la columna. Este tipo de estructuras son importantes para el tratamiento de datos que tienen una relación entre ellas, como tablas de resultados experimentales, sistemas de ecuaciones lineales o la representación de píxeles en imágenes digitales.

Los arreglos bidimensionales se almacenan en memoria de forma que siguen un orden de fila mayor. Este tipo de funciones en herramientas como el lenguaje de programación C, permiten una mayor optimización en el almacenamiento de datos, además de poder implementar algoritmos de mayor complejidad mediante el uso de ciclos anidados. En esta práctica se logrará desarrollar la capacidad lógica para manipular 2 índices de un arreglo bidimensional de manera simultánea, permitiendo recorrer, capturar y procesar información estructurada en tablas.

Desarrollo:

Programa 1a.c

A continuación, se observa un programa que genera un arreglo de dos dimensiones (arreglo multidimensional) y accede a cada uno de sus elementos por la posición que indica el renglón y la columna a través de dos ciclos for, uno anidado dentro de otro.



The image shows a code editor window with a file named 'main.c'. The code is a C program that prints a 3x3 matrix. The code is as follows:

```
1
2 #include<stdio.h>
3 int main()
4 {
5     int matriz[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
6     int i, j;
7     printf("Imprimir Matriz\n");
8     for (i=0 ; i<3 ; i++) //Representa al renglón del arreglo
9     {
10         for (j=0 ; j<3 ; j++)//Representa a la columna del arreglo
11         {
12             printf("%d, ",matriz[i][j]);
13         }
14         printf("\n");
15     }
16
17     return 0;
18 }
```

The console output shows the program's execution:

```
Imprimir Matriz
1, 2, 3,
4, 5, 6,
7, 8, 9,

...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 1: se observa la impresión de una matriz bidimensional de 3x3 utilizando ciclos **for** anidados.

Programa 2a.c

El siguiente programa genera un arreglo de dos dimensiones (arreglo multidimensional) y accede a sus elementos por la posición que indica el renglón y la columna a través de dos ciclos for, uno anidado dentro de otro, el contenido de cada elemento de este arreglo es la suma de sus índices.

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int i,j,a[5][5];
6     for (i=0 ; i<5 ; i++)//Representa al renglón del arreglo
7     {
8         for (j=0 ; j<5 ; j++)//Representa a la columna del arreglo
9         {
10             a[i][j]=i+j;
11             printf("\t%d, ",a[i][j]);
12         }
13         printf("\n");
14     }
15
16     return 0;
17 }
```

input

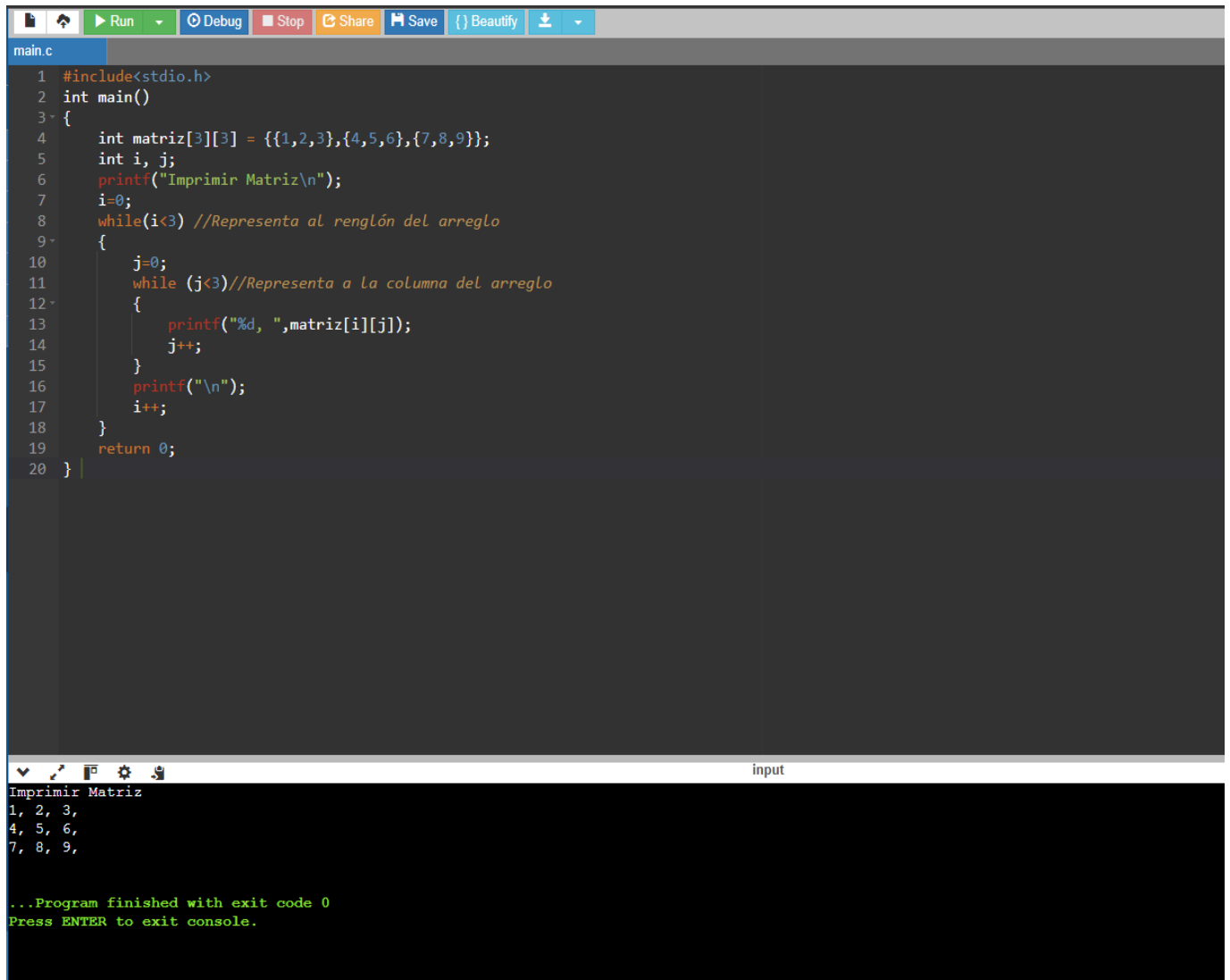
0,	1,	2,	3,	4,
1,	2,	3,	4,	5,
2,	3,	4,	5,	6,
3,	4,	5,	6,	7,
4,	5,	6,	7,	8,

...Program finished with exit code 0
Press ENTER to exit console.

Figura 2: se muestra una matriz de 5x5 generada mediante la suma de índices utilizando ciclos for.

Programa 1b.c

El código que se observa a continuación genera un arreglo de dos dimensiones (arreglo multidimensional) y accede a sus elementos por la posición que indica el renglón y la columna a través de dos ciclos while, uno anidado dentro de otro.

The image shows a code editor window with a file named 'main.c'. The code is a C program that prints a 3x3 matrix. It uses two nested 'while' loops to iterate through the rows and columns of the matrix. The matrix is initialized with the values {{1,2,3},{4,5,6},{7,8,9}}. The output in the terminal shows the matrix printed row by row, followed by a message indicating the program finished with exit code 0.

```
1 #include<stdio.h>
2 int main()
3 {
4     int matriz[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
5     int i, j;
6     printf("Imprimir Matriz\n");
7     i=0;
8     while(i<3) //Representa al renglón del arreglo
9     {
10         j=0;
11         while (j<3)//Representa a la columna del arreglo
12         {
13             printf("%d, ",matriz[i][j]);
14             j++;
15         }
16         printf("\n");
17         i++;
18     }
19     return 0;
20 }
```

Imprimir Matriz

1, 2, 3,

4, 5, 6,

7, 8, 9,

...Program finished with exit code 0

Press ENTER to exit console.

Figura 3: se aprecia la impresión de una matriz bidimensional utilizando ciclos while.

Programa 2b.c

Enseguida se muestra el código de un programa que permite generar un arreglo de dos dimensiones (arreglo multidimensional) y accede a sus elementos por la posición que indica el renglón y la columna a través de dos ciclos while, uno anidado dentro de otro, el contenido de cada elemento de este arreglo es la suma de sus índices.

The image shows a code editor window with a C program named `main.c`. The program uses nested `while` loops to generate a 5x5 matrix. The first loop iterates over rows (`i` from 0 to 4), and the second loop iterates over columns (`j` from 0 to 4). For each element `a[i][j]`, the value is calculated as `i+j` and printed. The output is displayed in a console window below the code editor, showing a 5x5 grid of values. The values in the first row are 0, 1, 2, 3, 4; in the second row 1, 2, 3, 4, 5; in the third row 2, 3, 4, 5, 6; in the fourth row 3, 4, 5, 6, 7; and in the fifth row 4, 5, 6, 7, 8. The program finishes with exit code 0.

```
1 #include<stdio.h>
2 int main()
3 {
4     int i,j,a[5][5];
5     i=0;
6     while (i<5) //Representa al renglón del arreglo
7     {
8         j=0;
9         while (j<5) //Representa a la columna del arreglo
10        {
11            a[i][j]=i+j;
12            printf("\t%d, ",a[i][j]);
13            j++;
14        }
15        printf("\n");
16        i++;
17    }
18    return 0;
19 }
```

input

0,	1,	2,	3,	4,
1,	2,	3,	4,	5,
2,	3,	4,	5,	6,
3,	4,	5,	6,	7,
4,	5,	6,	7,	8,

...Program finished with exit code 0
Press ENTER to exit console.

Figura 4: presenta una matriz de 5x5 cuyos valores fueron generados mediante la suma de índices usando ciclos while.

Programa 1c.c

A continuación, se presenta un código que permite la generación de un arreglo de dos dimensiones (arreglo multidimensional) y se puede acceder a cada uno de sus elementos por la posición que indica el renglón y la columna a través de dos ciclos do-while, uno anidado dentro de otro.

The image shows a code editor window with a toolbar at the top containing icons for Run, Debug, Stop, Share, Save, Beautify, and a download icon. The file name 'main.c' is visible in the tab. The code is as follows:

```
1  #include<stdio.h>
2  int main()
3  {
4      int matriz[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
5      int i, j;
6      printf("Imprimir Matriz\n");
7      i=0;
8
9      do //Representa al renglón del arreglo
10     {
11         j=0;
12         do //Representa a la columna del arreglo
13         {
14             printf("%d, ",matriz[i][j]);
15             j++;
16         }
17         while (j<3);
18         printf("\n");
19         i++;
20     }
21     while(i<3);
22     return 0;
23 }
```

Below the editor, a console window shows the output of the program:

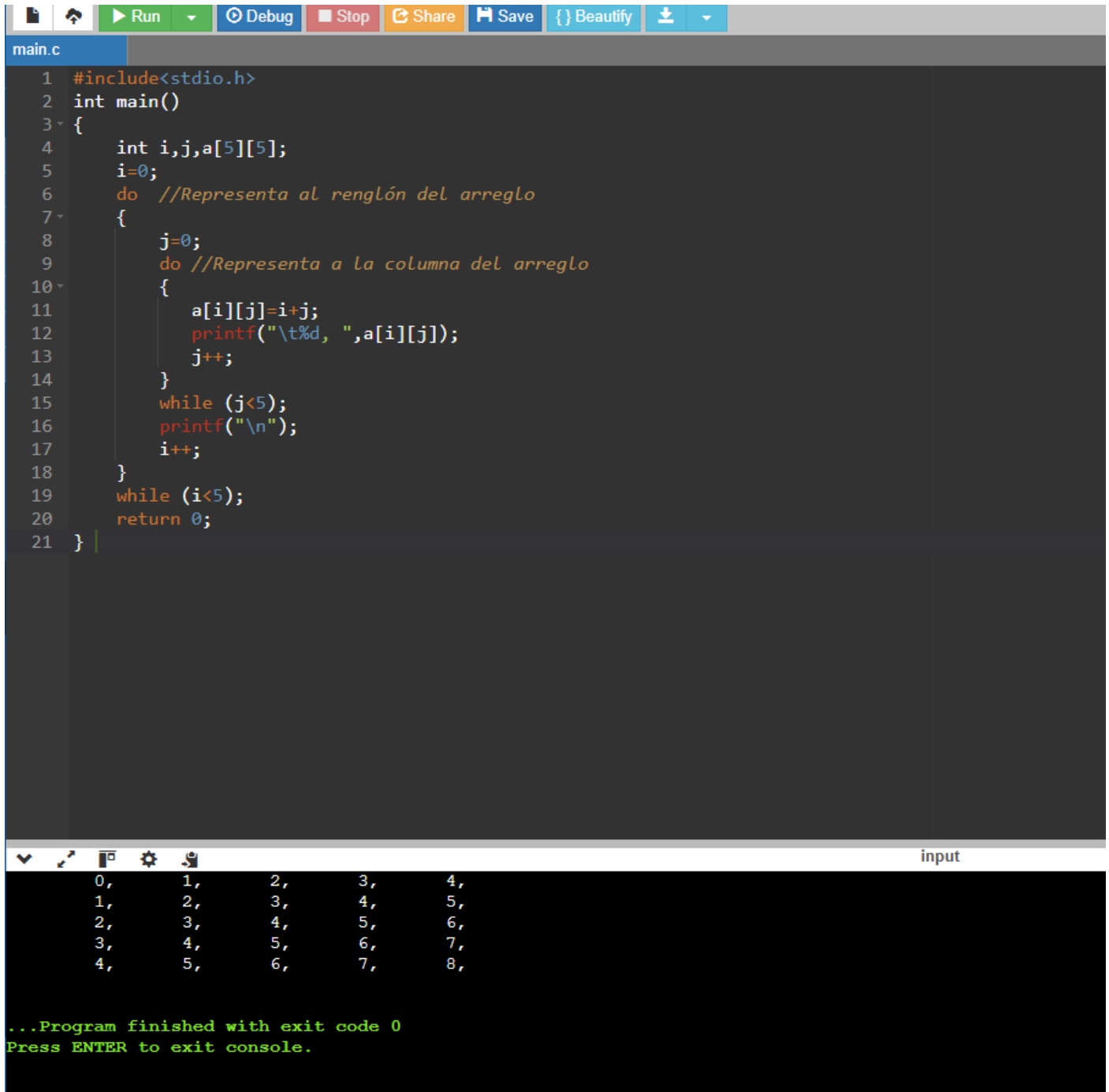
```
Imprimir Matriz
1, 2, 3,
4, 5, 6,
7, 8, 9,

...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 5: se observa la impresión de una matriz bidimensional utilizando ciclos `do-while`.

Programa 2c.c

Como puede observarse en el programa que se lee a continuación, se genera un arreglo de dos dimensiones (arreglo multidimensional) y se accede a todos sus elementos por la posición que indica el renglón y la columna a través de dos ciclos do-while, uno anidado dentro de otro, el contenido de cada uno de los elementos de este arreglo es la suma de sus índices.



```
main.c
1  #include<stdio.h>
2  int main()
3  {
4      int i,j,a[5][5];
5      i=0;
6      do //Representa al renglón del arreglo
7      {
8          j=0;
9          do //Representa a la columna del arreglo
10         {
11             a[i][j]=i+j;
12             printf("\t%d, ",a[i][j]);
13             j++;
14         }
15         while (j<5);
16         printf("\n");
17         i++;
18     }
19     while (i<5);
20     return 0;
21 }
```

input

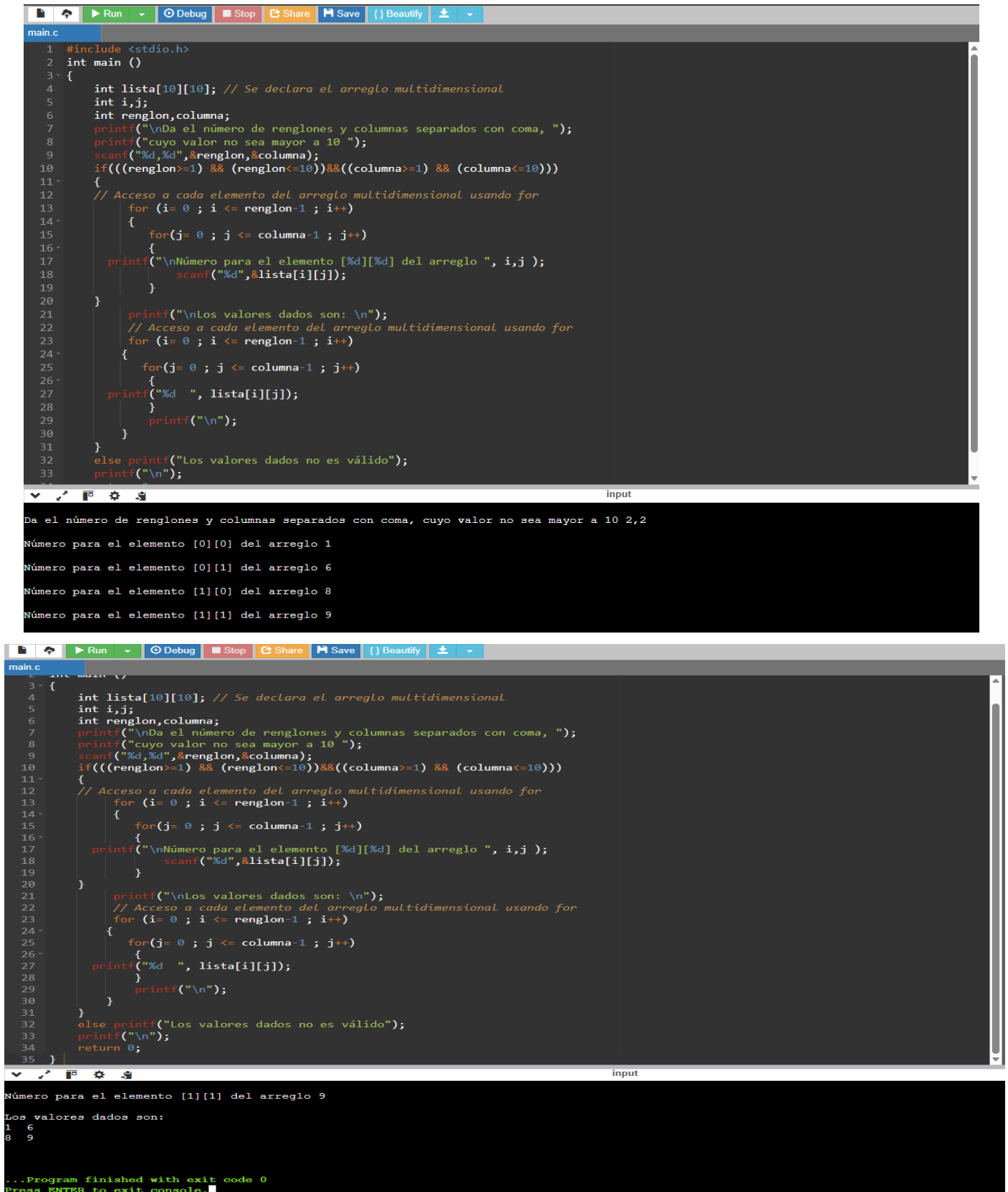
0,	1,	2,	3,	4,
1,	2,	3,	4,	5,
2,	3,	4,	5,	6,
3,	4,	5,	6,	7,
4,	5,	6,	7,	8,

...Program finished with exit code 0
Press ENTER to exit console.

Figura 6: muestra una matriz de 5x5 generada mediante la suma de índices utilizando ciclos do-while.

Programa 3.c

A continuación, se muestra un programa que genera un arreglo multidimensional de máximo 10 renglones y 10 columnas, para poder almacenar datos en cada elemento y posteriormente mostrar el contenido de esos elementos se hace uso de ciclos for anidados respectivamente.



```
main.c
1 #include <stdio.h>
2 int main ()
3 {
4     int lista[10][10]; // Se declara el arreglo multidimensional
5     int i,j;
6     int renglon,columna;
7     printf("\nDa el número de renglones y columnas separados con coma, ");
8     printf("cuyo valor no sea mayor a 10 ");
9     scanf("%d,%d",&renglon,&columna);
10    if(((renglon>=1) && (renglon<=10))&&((columna>=1) && (columna<=10)))
11    {
12        // Acceso a cada elemento del arreglo multidimensional usando for
13        for (i= 0 ; i <= renglon-1 ; i++)
14        {
15            for(j= 0 ; j <= columna-1 ; j++)
16            {
17                printf("\nNúmero para el elemento [%d][%d] del arreglo ", i,j );
18                scanf("%d",&lista[i][j]);
19            }
20        }
21        printf("\nLos valores dados son: \n");
22        // Acceso a cada elemento del arreglo multidimensional usando for
23        for (i= 0 ; i <= renglon-1 ; i++)
24        {
25            for(j= 0 ; j <= columna-1 ; j++)
26            {
27                printf("%d ", lista[i][j]);
28            }
29            printf("\n");
30        }
31    }
32    else printf("Los valores dados no es válido");
33    printf("\n");
34 }
35
```

input

Da el número de renglones y columnas separados con coma, cuyo valor no sea mayor a 10 2,2

Número para el elemento [0][0] del arreglo 1

Número para el elemento [0][1] del arreglo 6

Número para el elemento [1][0] del arreglo 8

Número para el elemento [1][1] del arreglo 9

```
main.c
1 #include <stdio.h>
2 int main ()
3 {
4     int lista[10][10]; // Se declara el arreglo multidimensional
5     int i,j;
6     int renglon,columna;
7     printf("\nDa el número de renglones y columnas separados con coma, ");
8     printf("cuyo valor no sea mayor a 10 ");
9     scanf("%d,%d",&renglon,&columna);
10    if(((renglon>=1) && (renglon<=10))&&((columna>=1) && (columna<=10)))
11    {
12        // Acceso a cada elemento del arreglo multidimensional usando for
13        for (i= 0 ; i <= renglon-1 ; i++)
14        {
15            for(j= 0 ; j <= columna-1 ; j++)
16            {
17                printf("\nNúmero para el elemento [%d][%d] del arreglo ", i,j );
18                scanf("%d",&lista[i][j]);
19            }
20        }
21        printf("\nLos valores dados son: \n");
22        // Acceso a cada elemento del arreglo multidimensional usando for
23        for (i= 0 ; i <= renglon-1 ; i++)
24        {
25            for(j= 0 ; j <= columna-1 ; j++)
26            {
27                printf("%d ", lista[i][j]);
28            }
29            printf("\n");
30        }
31    }
32    else printf("Los valores dados no es válido");
33    printf("\n");
34    return 0;
35 }
```

input

Número para el elemento [1][1] del arreglo 9

Los valores dados son:

1 6

8 9

...Program finished with exit code 0

Press ENTER to exit console.

Figura 7: se observa el ingreso y la visualización de datos almacenados en una matriz bidimensional.

Programa 4a.c

El programa siguiente genera un arreglo de dos dimensiones (arreglo multidimensional) y accede a sus elementos a través de un apuntador utilizando un ciclo for.



```
1 #include<stdio.h>
2 int main()
3 {
4     int matriz[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
5     int i, cont=0, *ap;
6     ap = *matriz; //Esta sentencia es análoga a: ap = &matriz[0][0];
7     printf("Imprimir Matriz\n");
8     for (i=0 ; i<3 ; i++)
9     {
10        if (cont == 3) //Se imprimió un renglón y se hace un salto de línea
11        {
12            printf("\n");
13            cont = 0; //Inicia conteo de elementos del siguiente renglón
14        }
15        printf("%d\t",*(ap+i)); //Se imprime el siguiente elemento de la matriz
16        cont++;
17    }
18    printf("\n");
19    return 0;
20 }
```

Imprimir Matriz

1	2	3
4	5	6
7	8	9

...Program finished with exit code 0
Press ENTER to exit console.

Figura 8: se presenta la impresión de una matriz bidimensional utilizando apuntadores y ciclos for.

Programa 4b.c

El código del siguiente programa genera un arreglo de dos dimensiones (arreglo multidimensional) y accede a sus elementos a través de un apuntador utilizando un ciclo while.

```
1 #include<stdio.h>
2 int main()
3 {
4     int matriz[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
5     int i, cont=0, *ap;
6     ap = *matriz; //Esta sentencia es análoga a: ap = &matriz[0][0];
7     printf("Imprimir Matriz\n");
8     i=0;
9     while (i<9)
10    {
11        if (cont == 3) //Se imprimió un renglón y se hace un salto de línea
12        {
13            printf("\n");
14            cont = 0; //Inicia conteo de elementos del siguiente renglón
15        }
16        printf("%d\t",*(ap+i)); //Se imprime el siguiente elemento de la matriz
17        cont++;
18        i++;
19    }
20    printf("\n");
21    return 0;
22 }
```

Imprimir Matriz

1	2	3
4	5	6
7	8	9

...Program finished with exit code 0
Press ENTER to exit console.

Figura 9: se aprecia el recorrido de una matriz bidimensional utilizando apuntadores y ciclos while.

Programa 4c.c

La generación de un arreglo de dos dimensiones (arreglo multidimensional) y el acceso a sus elementos a través de un apuntador utilizando un ciclo do-while se puede observar en el programa siguiente:

```
1 #include<stdio.h>
2 int main()
3 {
4     int matriz[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
5     int i, cont=0, *ap;
6     ap = *matriz; //Esta sentencia es análoga a: ap = &matriz[0][0];
7     printf("Imprimir Matriz\n");
8     i=0;
9     do
10    {
11        if (cont == 3) //Se imprimió un renglón y se hace un salto de línea
12        {
13            printf("\n");
14            cont = 0; //Inicia conteo de elementos del siguiente renglón
15        }
16        printf("%d\t",*(ap+i)); //Se imprime el siguiente elemento de la matriz
17        cont++;
18        i++;
19    } while (i<9);
20    printf("\n");
21    return 0;
22 }
```

Imprimir Matriz

1	2	3
4	5	6
7	8	9

...Program finished with exit code 0
Press ENTER to exit console.

Figura 10: muestra la impresión de una matriz bidimensional utilizando apuntadores y ciclos do-while.

Figura 11

Programa ejercicio 1 práctica 10

Ejercicio 1.

- a) En un arreglo se tienen registradas las ventas de cinco empleados durante cinco días de la semana. Se requiere determinar cuál fue la venta mayor realizada.

Nombre de la variable	Descripción	Tipo
I	Contador y subíndice	Entero
J	Contador y subíndice	Entero
V	Nombre del arreglo de ventas	Entero
VA	Representa la venta mayor realizada	Entero

Se tiene como objetivo desarrollar un algoritmo capaz de identificar la venta mayor registrada dentro de un conjunto de datos organizados en una estructura bidimensional. Para lograrlo, se emplea una matriz que permite almacenar las ventas de varios empleados a lo largo de varios días, optimizando así el manejo de múltiples datos bajo un mismo nombre de variable.

En primer lugar, se define una matriz denominada V, la cual tiene una dimensión de 5×5 , donde las filas representan a los empleados y las columnas corresponden a los días de la semana. El uso de un arreglo bidimensional es fundamental en este proceso, ya que permite organizar y acceder de manera eficiente a los datos registrados.

Posteriormente, se inicializan las variables necesarias para el control del flujo del algoritmo:

- Un acumulador denominado MA (Mayor), el cual almacenará la venta más alta encontrada dentro de la matriz. Este se inicializa con el primer valor del arreglo para asegurar una comparación válida desde el inicio.
- Dos variables de control o subíndices (i y j), que permiten recorrer las filas y columnas de la matriz de forma ordenada.
- Variables auxiliares (emp y dia), que se encargan de almacenar la posición (empleado y día) donde se encuentra la venta mayor.

Una vez definidas las variables, el algoritmo se ejecuta en dos fases principales mediante estructuras repetitivas:

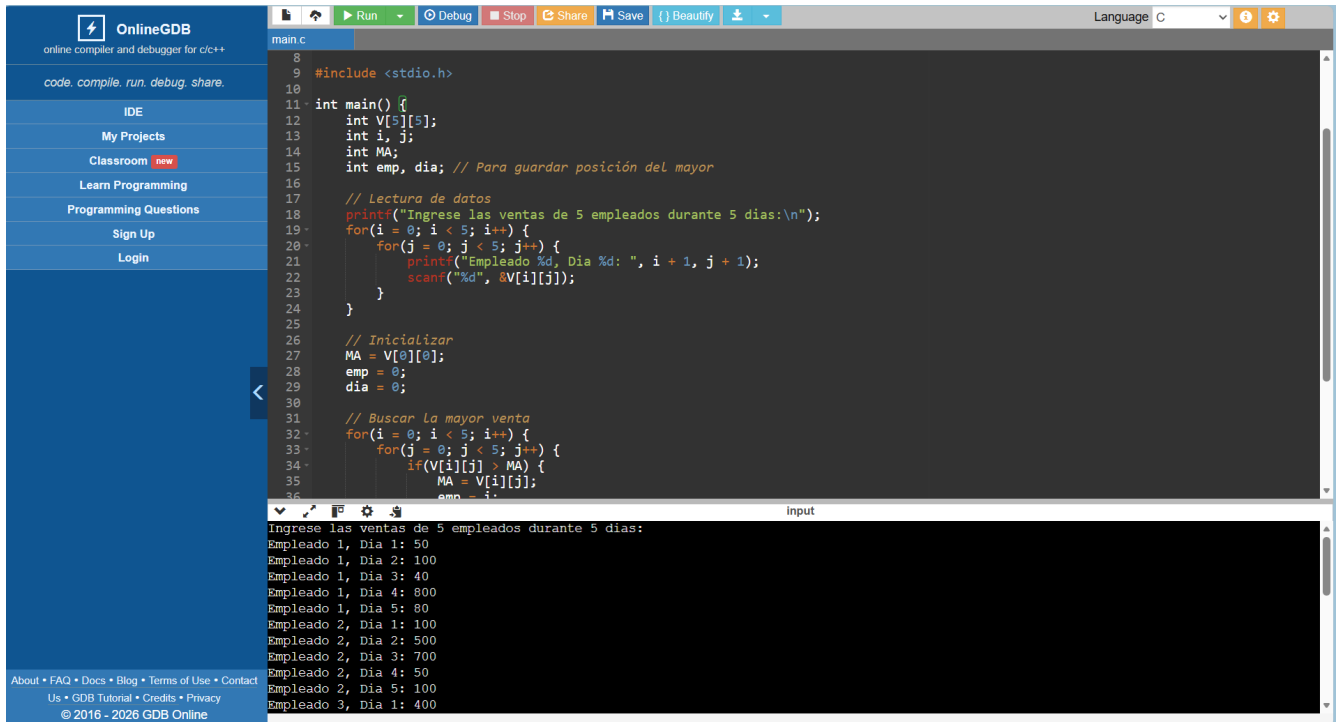
Fase de Captura:

Se utiliza un ciclo anidado para solicitar al usuario las ventas correspondientes a cada empleado durante cada día. Cada valor ingresado se almacena en la posición correspondiente de la matriz $V[i][j]$, donde los índices recorren desde 0 hasta 4 en ambas dimensiones.

Fase de Comparación:

Después de capturar los datos, se emplea otro conjunto de ciclos anidados para recorrer nuevamente la matriz. En cada iteración, se compara el valor actual con el contenido de la variable MA. Si el valor es mayor, se actualiza MA y se registran las posiciones correspondientes en las variables emp y dia.

Finalmente, el algoritmo muestra como salida la venta mayor encontrada, así como el empleado y el día en que se realizó dicha venta, proporcionando así una interpretación clara de los resultados obtenidos.

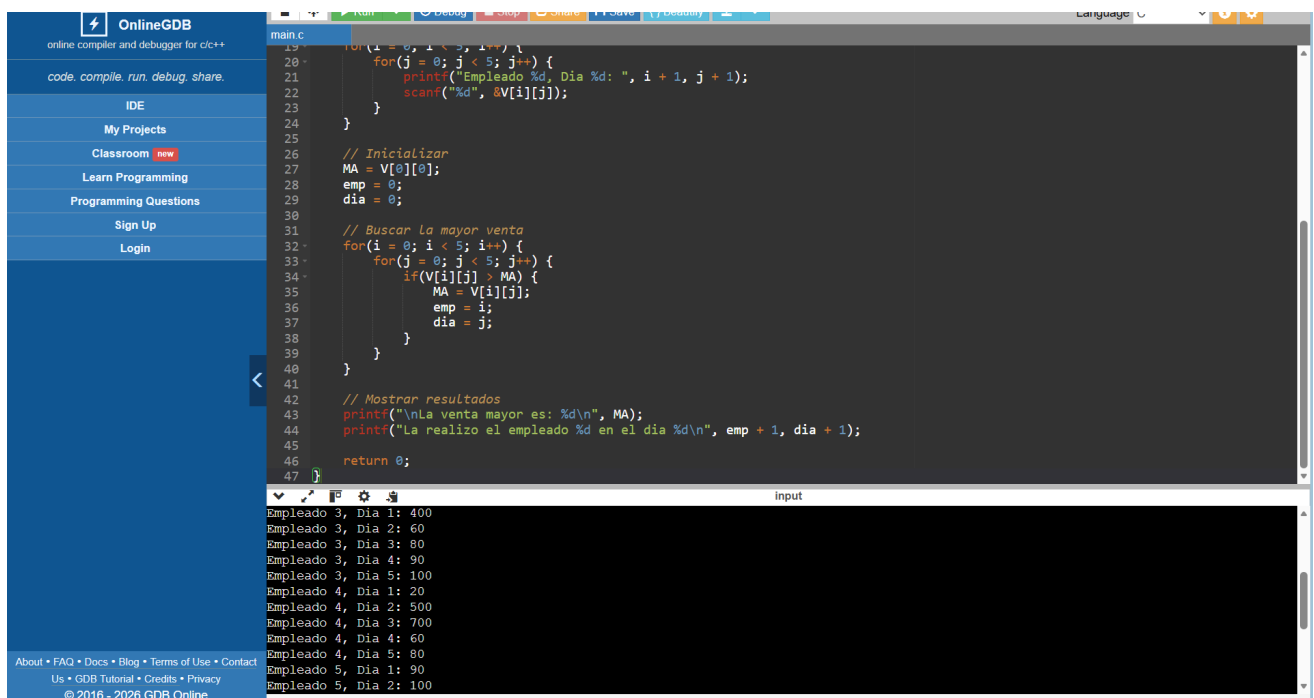


```
main.c
8
9 #include <stdio.h>
10
11 int main() {
12     int V[5][5];
13     int i, j;
14     int MA;
15     int emp, dia; // Para guardar posición del mayor
16
17     // Lectura de datos
18     printf("Ingrese las ventas de 5 empleados durante 5 dias:\n");
19     for(i = 0; i < 5; i++) {
20         for(j = 0; j < 5; j++) {
21             printf("Empleado %d, Dia %d: ", i + 1, j + 1);
22             scanf("%d", &V[i][j]);
23         }
24     }
25
26     // Inicializar
27     MA = V[0][0];
28     emp = 0;
29     dia = 0;
30
31     // Buscar la mayor venta
32     for(i = 0; i < 5; i++) {
33         for(j = 0; j < 5; j++) {
34             if(V[i][j] > MA) {
35                 MA = V[i][j];
36                 emp = i;
37                 dia = j;
38             }
39         }
40     }
41
42     // Mostrar resultados
43     printf("\nLa venta mayor es: %d\n", MA);
44     printf("La realizo el empleado %d en el dia %d\n", emp + 1, dia + 1);
45     return 0;
46 }
```

Input

Ingrese las ventas de 5 empleados durante 5 dias:

Empleado 1, Dia 1: 50
Empleado 1, Dia 2: 100
Empleado 1, Dia 3: 40
Empleado 1, Dia 4: 800
Empleado 1, Dia 5: 80
Empleado 2, Dia 1: 100
Empleado 2, Dia 2: 500
Empleado 2, Dia 3: 700
Empleado 2, Dia 4: 50
Empleado 2, Dia 5: 100
Empleado 3, Dia 1: 400



```
main.c
19
20     for(i = 0; i < 5; i++) {
21         for(j = 0; j < 5; j++) {
22             printf("Empleado %d, Dia %d: ", i + 1, j + 1);
23             scanf("%d", &V[i][j]);
24         }
25     }
26
27     // Inicializar
28     MA = V[0][0];
29     emp = 0;
30     dia = 0;
31
32     // Buscar la mayor venta
33     for(i = 0; i < 5; i++) {
34         for(j = 0; j < 5; j++) {
35             if(V[i][j] > MA) {
36                 MA = V[i][j];
37                 emp = i;
38                 dia = j;
39             }
40         }
41     }
42
43     // Mostrar resultados
44     printf("\nLa venta mayor es: %d\n", MA);
45     printf("La realizo el empleado %d en el dia %d\n", emp + 1, dia + 1);
46     return 0;
47 }
```

Input

Empleado 3, Dia 1: 400
Empleado 3, Dia 2: 60
Empleado 3, Dia 3: 80
Empleado 3, Dia 4: 90
Empleado 3, Dia 5: 100
Empleado 4, Dia 1: 20
Empleado 4, Dia 2: 500
Empleado 4, Dia 3: 700
Empleado 4, Dia 4: 60
Empleado 4, Dia 5: 80
Empleado 5, Dia 1: 90
Empleado 5, Dia 2: 100

```
main.c
19 for(i = 0; i < 5; i++) {
20     for(j = 0; j < 5; j++) {
21         printf("Empleado %d, Dia %d: ", i + 1, j + 1);
22         scanf("%d", &V[i][j]);
23     }
24 }
25
26 // Inicializar
27 MA = V[0][0];
28 emp = 0;
29 dia = 0;
30
31 // Buscar la mayor venta
32 for(i = 0; i < 5; i++) {
33     for(j = 0; j < 5; j++) {
34         if(V[i][j] > MA) {
35             MA = V[i][j];
36             emp = i;
37             dia = j;
38         }
39     }
40 }
41
42 // Mostrar resultados
43 printf("\nLa venta mayor es: %d\n", MA);
44 printf("La realizo el empleado %d en el dia %d\n", emp + 1, dia + 1);
45
46 return 0;
47 }
```

input

```
Empleado 5, Dia 1: 90
Empleado 5, Dia 2: 100
Empleado 5, Dia 3: 500
Empleado 5, Dia 4: 450
Empleado 5, Dia 5: 900

La venta mayor es: 900
La realizo el empleado 5 en el dia 5

...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 11: Se muestra la ejecución del algoritmo que permite almacenar las ventas en una matriz bidimensional y determinar la venta máxima mediante un proceso de comparación iterativa, indicando su ubicación dentro de la estructura.

Figura 12

Programa ejercicio 2 práctica 10

Ejercicio 2.

- b) Realice el algoritmo para obtener una matriz como el resultado de la suma de dos matrices de orden M x N.

Nombre de la variable	Descripción	Tipo
I	Contador y subíndice	Entero
J	Contador y subíndice	Entero
A, B	Nombres de los arreglos por sumar	Entero
C	Nombre del arreglo resultante	Entero

M	Número de renglones del arreglo	Entero
N	Número de columnas del arreglo	Entero

Se tiene como objetivo desarrollar un algoritmo capaz de obtener una matriz resultante a partir de la suma de dos conjuntos de datos numéricos organizados en estructuras bidimensionales. Para lograrlo, se emplean arreglos multidimensionales (matrices), los cuales permiten manipular múltiples valores organizados en filas y columnas bajo un mismo nombre de variable.

En primer lugar, se definen dos matrices denominadas A y B, las cuales tienen dimensiones $M \times N$, donde M representa el número de renglones y N el número de columnas. Asimismo, se define una tercera matriz C, que almacenará el resultado de la suma de las matrices anteriores. El uso de estas estructuras es fundamental, ya que permite realizar operaciones elemento a elemento de manera eficiente.

Posteriormente, se inicializan las variables necesarias para el control del flujo:

- Dos variables de control o subíndices (i y j), que permiten recorrer las filas y columnas de las matrices.
- Dos variables (M y N), que indican las dimensiones de las matrices.
- Tres arreglos bidimensionales (A, B y C), donde los dos primeros almacenan los datos de entrada y el último guarda el resultado.

Una vez configuradas las variables, el algoritmo se ejecuta en tres fases principales mediante estructuras repetitivas:

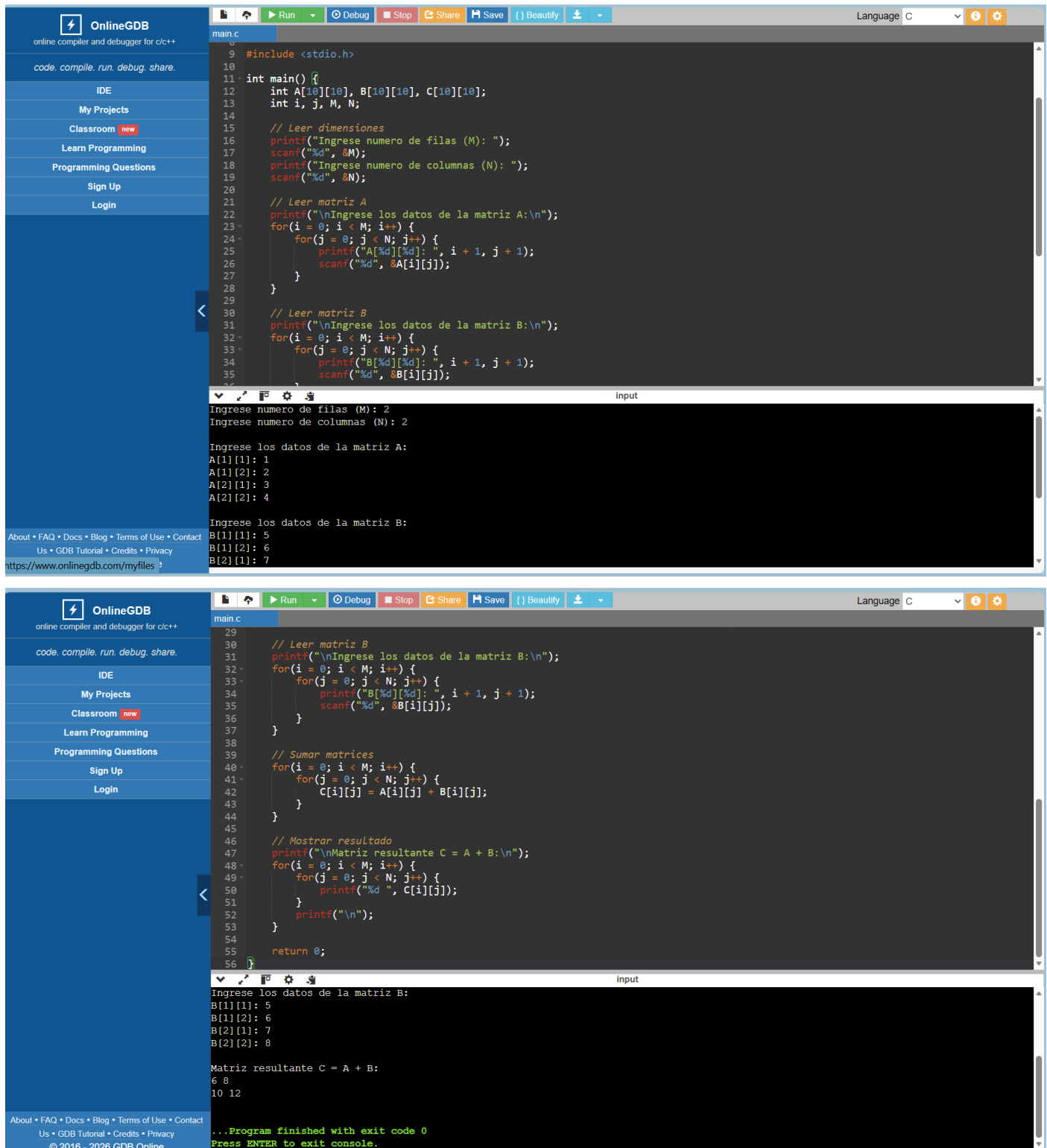
- Fase de Captura:
Se utilizan ciclos anidados para solicitar al usuario los valores de las matrices A y B. Cada elemento ingresado se almacena en la posición correspondiente $A[i][j]$ y $B[i][j]$, recorriendo desde 0 hasta $M-1$ en filas y de 0 hasta $N-1$ en columnas.
- Fase de Procesamiento (Suma):
Se emplean nuevamente ciclos anidados para recorrer ambas matrices simultáneamente. En cada iteración, se realiza la suma de los elementos correspondientes:

$$C[i][j] = A[i][j] + B[i][j]$$

El resultado se almacena en la matriz C en la misma posición.

- Fase de Salida:
- Finalmente, se recorren los elementos de la matriz C para mostrar en pantalla la matriz resultante, organizada en forma de filas y columnas.

De esta manera, el algoritmo permite obtener de forma clara y ordenada la suma de dos matrices del mismo tamaño, facilitando la manipulación de datos estructurados en dos dimensiones.



The image displays two screenshots of the OnlineGDB IDE, illustrating the execution of a C program for matrix addition.

Top Screenshot: The code defines two 2x2 matrices, A and B, and calculates their sum, C. The input phase shows the user entering the dimensions (M=2, N=2) and the elements of matrix A and B.

```
#include <stdio.h>

int main() {
    int A[10][10], B[10][10], C[10][10];
    int i, j, M, N;

    // Leer dimensiones
    printf("Ingrese numero de filas (M): ");
    scanf("%d", &M);
    printf("Ingrese numero de columnas (N): ");
    scanf("%d", &N);

    // Leer matriz A
    printf("\nIngrese los datos de la matriz A:\n");
    for(i = 0; i < M; i++) {
        for(j = 0; j < N; j++) {
            printf("A[%d][%d]: ", i + 1, j + 1);
            scanf("%d", &A[i][j]);
        }
    }

    // Leer matriz B
    printf("\nIngrese los datos de la matriz B:\n");
    for(i = 0; i < M; i++) {
        for(j = 0; j < N; j++) {
            printf("B[%d][%d]: ", i + 1, j + 1);
            scanf("%d", &B[i][j]);
        }
    }

    // Sumar matrices
    for(i = 0; i < M; i++) {
        for(j = 0; j < N; j++) {
            C[i][j] = A[i][j] + B[i][j];
        }
    }

    // Mostrar resultado
    printf("\nMatriz resultante C = A + B:\n");
    for(i = 0; i < M; i++) {
        for(j = 0; j < N; j++) {
            printf("%d ", C[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Bottom Screenshot: The program has finished execution, displaying the resulting matrix C. The output shows the elements of matrix B and the resulting matrix C.

```
Matriz resultante C = A + B:
6 8
10 12

...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 12 : Se presenta la ejecución del algoritmo de suma de matrices, en el cual se realiza la captura de datos y el procesamiento mediante operaciones elemento a elemento, generando una matriz resultante con los valores obtenidos.

Análisis de resultados:

Programa 1a.c

El programa tiene como finalidad mostrar en pantalla los elementos de una matriz bidimensional de 3x3 previamente inicializada. Para ello, se declara un arreglo llamado `matriz` que almacena números enteros organizados en renglones y columnas. Posteriormente, mediante dos ciclos `for` anidados, el programa recorre cada posición del arreglo utilizando las variables `i` y `j`, donde `i` representa los renglones y `j` las columnas. Cada elemento es impreso en pantalla usando `printf`, permitiendo visualizar toda la matriz de forma ordenada. Finalmente, después de imprimir cada renglón, se realiza un salto de línea para mejorar la presentación de los datos.

Programa 1b.c

El objetivo de este programa es imprimir los elementos de una matriz bidimensional utilizando ciclos `while` en lugar de ciclos `for`. La matriz de 3x3 contiene valores enteros definidos previamente y es recorrida mediante dos ciclos `while` anidados. La variable `i` controla los renglones mientras que `j` controla las columnas. Conforme avanzan los ciclos, el programa accede a cada elemento de la matriz y lo muestra en pantalla. Después de completar cada renglón, se imprime un salto de línea para conservar el formato matricial. Este código demuestra que el acceso a arreglos multidimensionales puede realizarse utilizando diferentes estructuras repetitivas.

Programa 1c.c

El programa tiene como propósito imprimir una matriz de 3x3 utilizando ciclos `do-while`. La matriz contiene valores enteros inicializados previamente y es recorrida mediante dos ciclos anidados. A diferencia de otros ciclos, el `do-while` ejecuta primero las instrucciones y después verifica la condición, garantizando que el bloque de código se ejecute al menos una vez. El programa utiliza la variable `i` para controlar los renglones y `j` para las columnas, permitiendo acceder a cada elemento de la matriz e imprimirlo en pantalla de manera ordenada. Además, se realiza un salto de línea después de cada renglón para facilitar la lectura de la matriz.

Programa 2a.c

Este programa genera y muestra una matriz de 5x5 utilizando arreglos multidimensionales y ciclos `for`. A diferencia del código anterior, los valores de la matriz no están definidos desde el inicio, sino que se calculan automáticamente mediante la suma de los índices `i` y `j`. El programa utiliza dos ciclos `for` anidados para recorrer cada posición del arreglo y almacenar el resultado de la suma en cada elemento. Después de asignar el valor correspondiente, este se imprime inmediatamente en pantalla. Gracias a esta lógica, se obtiene una matriz cuyos valores aumentan de acuerdo con la posición de cada elemento dentro del arreglo.

Programa 2b.c

Este programa crea una matriz de 5x5 utilizando ciclos `while` para recorrer sus posiciones. El contenido de cada elemento se obtiene mediante la suma de los índices correspondientes al renglón y la columna. Primero, la variable `i` controla el recorrido de los renglones y, dentro de este proceso, la variable `j` permite recorrer las columnas. Cada resultado de la suma se almacena en la matriz y posteriormente se imprime en pantalla. El programa permite comprender cómo funcionan los arreglos bidimensionales utilizando estructuras `while`, manteniendo la misma lógica de acceso y manipulación de datos.

Programa 2c.c

Este código genera una matriz de 5x5 utilizando ciclos `do-while` anidados. Cada elemento del arreglo almacena la suma de sus índices, es decir, del número de renglón y columna correspondientes. El programa recorre toda la matriz utilizando las variables `i` y `j`, asignando y mostrando cada valor conforme avanza el recorrido. La estructura `do-while` asegura que cada sección del código se ejecute por lo menos una vez antes de validar la condición de repetición. El resultado final es una matriz organizada cuyos valores dependen de la posición de cada elemento.

Programa 3.c

El programa permite al usuario crear y manipular una matriz bidimensional con un tamaño máximo de 10 renglones y 10 columnas. Inicialmente, el usuario introduce la cantidad de renglones y columnas que desea utilizar, y el programa verifica que dichos valores se encuentren dentro del rango permitido. Posteriormente, mediante ciclos `for` anidados, se solicita al usuario ingresar los valores de cada posición de la matriz. Una vez capturados todos los datos, el programa vuelve a recorrer el arreglo para mostrar en pantalla los elementos almacenados. Este código demuestra el uso práctico de arreglos multidimensionales para almacenar información ingresada dinámicamente por el usuario.

Programa 4a.c

El propósito de este programa es recorrer e imprimir una matriz bidimensional utilizando apuntadores y un ciclo `for`. La matriz de 3x3 se encuentra inicializada con valores enteros y posteriormente se declara un apuntador llamado `ap`, el cual almacena la dirección del primer elemento de la matriz. El programa utiliza aritmética de apuntadores para acceder a cada posición del arreglo mediante la expresión `*(ap+i)`. Además, se utiliza la variable `cont` para controlar la impresión de los renglones y realizar saltos de línea cada tres elementos. Este código demuestra cómo los arreglos multidimensionales pueden recorrerse de manera continua en memoria utilizando apuntadores.

Programa 4b.c

Este programa imprime una matriz bidimensional utilizando apuntadores y ciclos `while`. Primero, se declara una matriz de 3x3 y un apuntador que almacena la dirección del primer elemento del arreglo. A través de un ciclo `while`, el programa recorre secuencialmente todos los elementos de la matriz utilizando aritmética de apuntadores. La variable `cont` permite identificar cuándo se han mostrado tres elementos, generando un salto de línea para mantener el formato matricial. El programa evidencia la relación existente entre los arreglos y los apuntadores en el lenguaje C, mostrando cómo se puede acceder a los datos almacenados en memoria de forma directa.

Programa 4c.c

El programa tiene como finalidad recorrer e imprimir una matriz bidimensional utilizando apuntadores y ciclos `do-while`. La matriz contiene valores enteros previamente definidos y el apuntador `ap` almacena la dirección del primer elemento del arreglo. Posteriormente, mediante un ciclo `do-while`, el programa accede a cada elemento utilizando aritmética de apuntadores y muestra los datos en pantalla. La variable `cont` controla la cantidad de elementos impresos por renglón para conservar el formato de la matriz. Este código permite comprender cómo los apuntadores pueden utilizarse para recorrer arreglos multidimensionales de manera eficiente utilizando estructuras repetitivas `do-while`.

Programa ejercicio 1 práctica 10

A partir de la ejecución del programa, se pudo comprobar que el algoritmo cumple correctamente con el objetivo de identificar la venta mayor dentro de un conjunto de datos organizados en una matriz bidimensional. La estructura utilizada permitió representar de manera clara la información correspondiente a las ventas de los empleados a lo largo de varios días, facilitando su manipulación y análisis.

Durante la fase de captura, los datos fueron ingresados correctamente en cada una de las posiciones de la matriz, lo que garantizó que la información estuviera completa y organizada. Posteriormente, en la fase de procesamiento, el uso de ciclos anidados permitió recorrer la totalidad de la matriz sin omitir ningún elemento, asegurando así una evaluación exhaustiva de todos los valores registrados.

El proceso de comparación demostró ser eficiente, ya que en cada iteración se evaluó si el valor actual superaba al previamente almacenado como máximo. En caso afirmativo, se actualizó la variable correspondiente, lo que permitió mantener en todo momento el control del valor más alto.

encontrado. Asimismo, el uso de variables auxiliares permitió registrar con precisión la posición de dicho valor, identificando el empleado y el día en que se realizó la venta mayor.

Los resultados obtenidos evidencian que el algoritmo funciona de manera correcta y consistente, siempre que los datos de entrada sean válidos. Además, se destaca que este tipo de solución es escalable, ya que podría adaptarse fácilmente a matrices de mayor tamaño sin modificar su lógica principal. En general, el programa demuestra la utilidad de los arreglos bidimensionales y de las estructuras repetitivas para la resolución de problemas que implican búsqueda y comparación de datos.

Programa ejercicio 2 práctica 10

La ejecución del programa permitió verificar que el algoritmo realiza de manera correcta la suma de dos matrices de dimensiones $M \times N$. Desde la fase inicial, el usuario pudo definir el tamaño de las matrices, lo cual aporta flexibilidad al programa y permite adaptarlo a diferentes conjuntos de datos.

Durante la fase de captura, los valores fueron almacenados adecuadamente en las matrices A y B, respetando la estructura de filas y columnas. Esto aseguró que cada elemento tuviera una correspondencia directa entre ambas matrices, condición indispensable para realizar la operación de suma correctamente.

En la fase de procesamiento, el uso de ciclos anidados permitió recorrer ambas matrices de manera simultánea, efectuando la suma de los elementos correspondientes en cada posición. Este procedimiento garantizó que cada elemento de la matriz resultante fuera calculado correctamente mediante la operación $C[i][j] = A[i][j] + B[i][j]$.

Posteriormente, en la fase de salida, se mostraron los resultados de forma organizada, permitiendo visualizar claramente la matriz resultante. Los valores obtenidos coinciden con lo esperado matemáticamente, lo que confirma la validez del algoritmo implementado.

En conclusión, los resultados demuestran que el programa es eficiente y confiable para realizar operaciones entre matrices del mismo tamaño. Además, resalta la importancia del uso de arreglos multidimensionales y estructuras repetitivas en la resolución de problemas que involucran grandes volúmenes de datos organizados. Este tipo de algoritmos constituye una base fundamental para aplicaciones más avanzadas en el manejo de información matricial.

Conclusión:

Macay Martínez David: Tras el desarrollo de la práctica, se concluyó que los arreglos bidimensionales son herramientas de almacenamiento importantes para programar, ya que proporcionan una estructura lógica que se alinea con la organización de diversos datos en la ingeniería. A comparación de los arreglos unidimensionales, el uso de matrices facilita la representación de sistemas complejos donde la información depende de 2 variables.

A través del desarrollo de los programas, se demostró que el manejo de 2 índices es la base del control de estructuras. Se comprendió que el primer índice maneja el desplazamiento de filas y el segundo el horizontal, este orden es importante de respetar para implementar ciclos anidados.

Finalmente, se visualizaron los datos ingresados en pantalla en forma de tabla, a pesar de que el compilador los almacena de forma lineal. La elaboración de esta práctica permite reforzar habilidades para decidir en que tipo de situaciones una estructura bidimensional es la solución más óptima.

Nolasco Ramírez Sofia:

En el desarrollo de la presente práctica se cumplió satisfactoriamente el objetivo de emplear arreglos bidimensionales para la resolución de problemas que requieren el agrupamiento de datos homogéneos mediante estructuras de dos índices. A través de diversos ejercicios, se analizaron las metodologías fundamentales para la declaración, inicialización y recorrido de matrices en el lenguaje C, permitiendo comprender su correcta implementación.

Se comprobó la versatilidad de los arreglos multidimensionales al ser manipulados mediante distintas estructuras de control repetitivas. En particular, se evidenció que es posible acceder y procesar cada elemento de una matriz utilizando ciclos for, while y do-while anidados, obteniendo resultados consistentes en la organización y visualización de la información. Asimismo, se exploró el uso de la aritmética de apuntadores como una alternativa eficiente para el acceso continuo a la memoria, lo cual permitió profundizar en la relación existente entre los punteros y las estructuras de datos bidimensionales.

La aplicación práctica de estos conceptos se consolidó mediante la resolución de algoritmos específicos, tales como la identificación de valores máximos dentro de un conjunto de datos y la ejecución de operaciones matemáticas elemento a elemento, como la suma de matrices de orden $M \times N$. Dichos ejercicios permitieron verificar que el uso de ciclos anidados garantiza un recorrido completo de la información, asegurando la precisión y confiabilidad de los resultados obtenidos.

En conclusión, se establece que el dominio de los arreglos multidimensionales, en conjunto con el uso adecuado de las estructuras de control, resulta fundamental para la gestión eficiente de grandes volúmenes de datos estructurados, constituyendo una base sólida para el desarrollo de programas más complejos, robustos y escalables.

Bibliografía

El lenguaje de programación C. Brian W. Kernighan, Dennis M. Ritchie, segunda edición, USA, Pearson Educación 1991.